22/09/2026

Conception — Remise 2 (prototype local)

Projet SmartDisplay

Cedric Tresor Tchipkap Tchuigoua
CEGEP DE RIVIERE-DU-LOUP

Table des matières

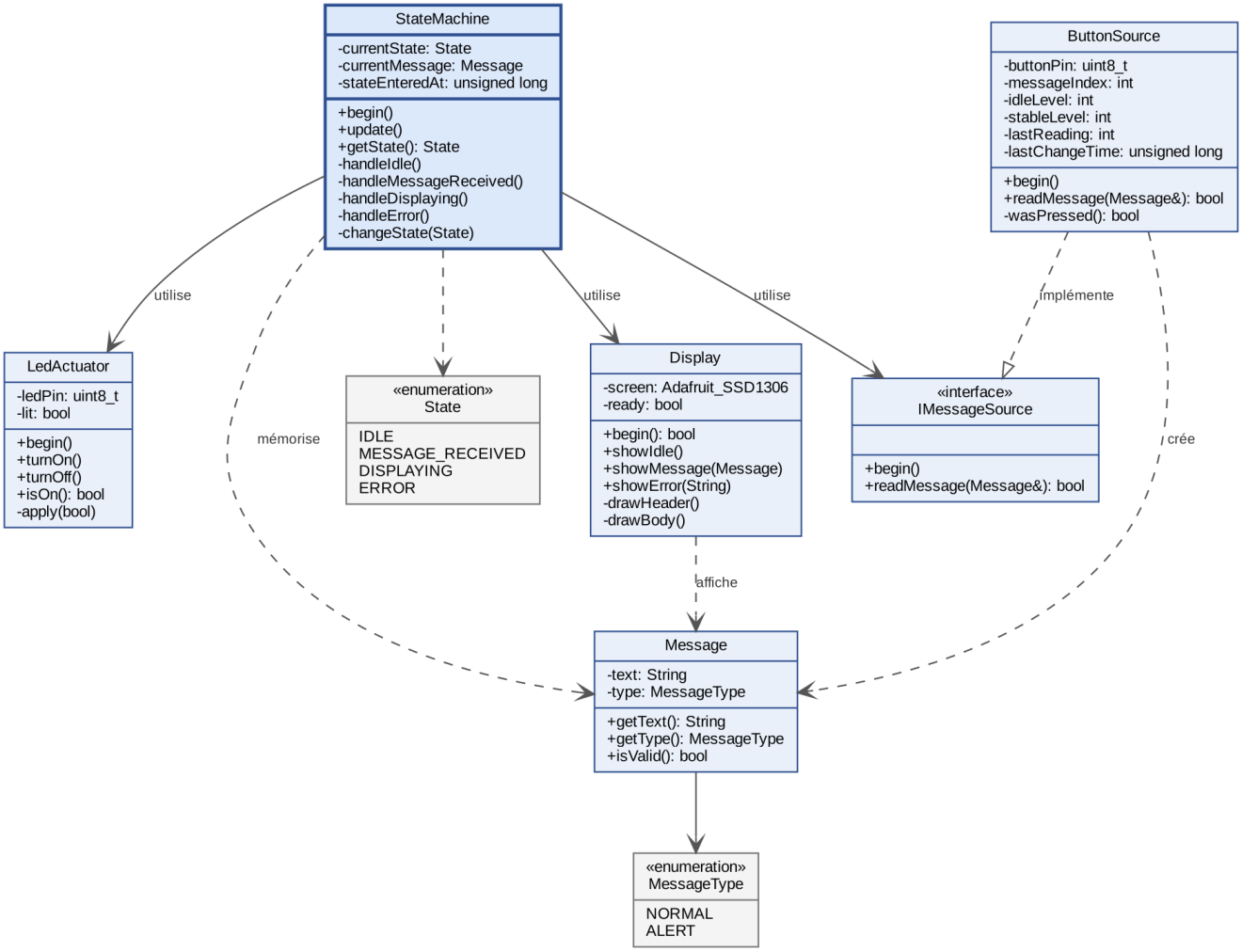
1. Introduction 2
2. Architecture orientée objet 2
3. Rôle des classes 2
4. Machine à états (version partielle) 3
   a. Transitions 3
   b. Écarts avec la machine à états de la Remise 1 4
   c. Correspondance des noms 4
5. Matériel et câblage 4
6. Scénario de test manuel 5
7. Utilisation de l'intelligence artificielle 5

1. Introduction

Ce document décrit l'architecture du prototype local de la Remise 2 : l'ESP32 fonctionne seul, avec un bouton qui simule la réception de messages, un écran OLED et une DEL d'alerte. Le Wi-Fi, MQTT, Node-RED, InfluxDB et Grafana sont ajoutés à la Remise 3.

2. Architecture orientée objet

Chaque classe a une seule responsabilité. Le fichier main.cpp se limite à créer les objets, à les initialiser et à appeler la machine à états à chaque tour de loop() : il ne contient aucune logique métier.



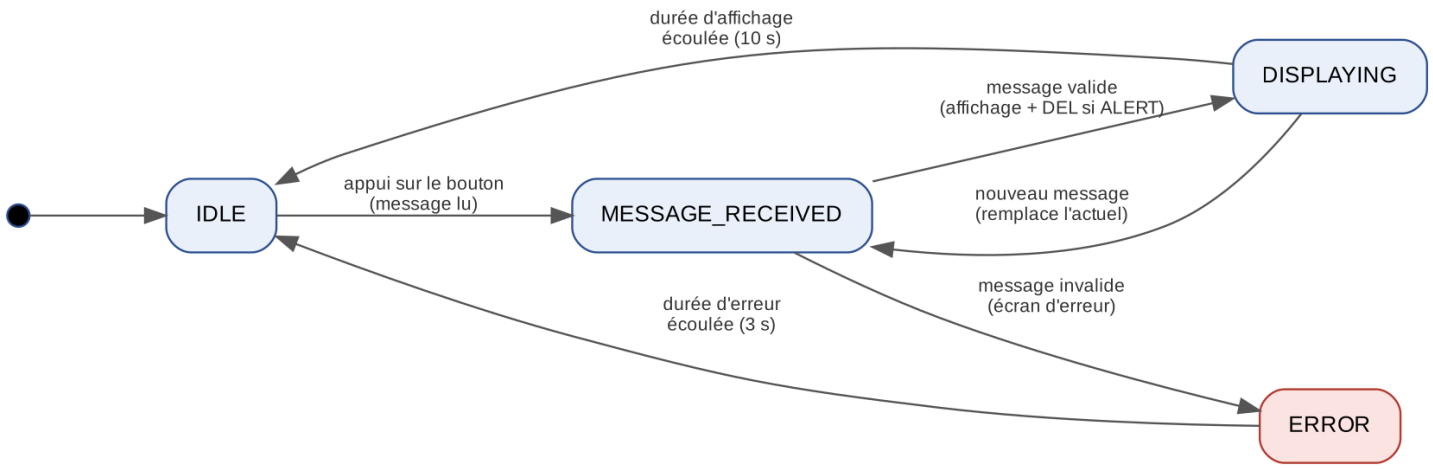
3. Rôle des classes

Classe	Dossier	Responsabilité
Message	models	Contient le texte et le type (NORMAL ou ALERT) ; vérifie qu'un message est valide (non vide, 60 caractères maximum).

Classe	Dossier	Responsabilité
IMessageSource	interfaces	Contrat de toute source de messages : begin() et readMessage(), une lecture non bloquante.
ButtonSource	inputs	Lit le bouton avec anti-rebond (millis()) et fournit à chaque appui un message de test.
Display	outputs	Affiche l'état sur l'écran OLED, avec une icône 24x24 pour l'attente, l'information, l'alerte et l'erreur.
LedActuator	outputs	Allume et éteint la DEL rouge d'alerte.
StateMachine	core	Coordonne la source, l'affichage et la DEL selon l'état courant ; reste non bloquante.
Config	config	Regroupe les broches et les constantes (durées, adresse I2C, anti-rebond).

Choix de conception : la machine à états ne connaît que l'interface IMessageSource, pas le bouton. ***À la Remise 3****, une classe de source MQTT implémentera la même interface et remplacera le bouton sans modifier la machine à états.*

4. Machine à états (version partielle)



a. Transitions

De	Vers	Condition	Actions
IDLE	MESSAGE_RECEIVED	Un appui est détecté (message lu)	Aucune
MESSAGE_RECEIVED	DISPLAYING	Message valide	Affichage du message ; DEL allumée si ALERT, éteinte sinon
MESSAGE_RECEIVED	ERROR	Message invalide	Écran d'erreur ; DEL éteinte

De	Vers	Condition	Actions
DISPLAYING	MESSAGE_RECEIVED	Nouveau message	Le nouveau message remplace l'actuel (un seul message à la fois)
DISPLAYING	IDLE	Durée d'affichage écoulée (10 s)	DEL éteinte ; écran d'attente
ERROR	IDLE	Durée d'erreur écoulée (3 s)	Écran d'attente

b. Écarts avec la machine à états de la Remise 1

- ACCUSE_RECEPTION et CONFIGURATION ne sont pas encore implantés : ils s'ajoutent à la Remise 3, avec MQTT.
- Sans accusé de réception, DISPLAYING retourne directement à IDLE. À la Remise 3, l'état ACK_SENT s'insère entre les deux.
- La transition DISPLAYING vers MESSAGE_RECEIVED (remplacement d'un message) et le retour automatique à IDLE après un délai sont nouveaux. Le diagramme de la Remise 1 sera mis à jour à la Remise 4.

c. Correspondance des noms

Remise 1 (français)	Code (anglais)
ATTENTE	IDLE
MESSAGE_RECU	MESSAGE_RECEIVED
AFFICHAGE	DISPLAYING
ERREUR	ERROR
ACCUSE_RECEPTION (Remise 3)	ACK_SENT
CONFIGURATION (Remise 3)	CONFIGURING

5. Matériel et câblage

Composant	Broche ESP32	Remarques
DEL rouge	GPIO 26	Broche, puis résistance de 220 à 330 Ω, puis patte longue (anode) ; patte courte (cathode) au GND.
Bouton	GPIO 35	Entrée seulement, sans pull-up ni pull-down interne : résistance externe de 10 kΩ vers 3,3 V ou vers GND. Le code détecte le niveau de repos au démarrage.
Écran OLED SSD1306 128x64 (SDA)	GPIO 21	Bus I2C, adresse 0x3C (parfois 0x3D).
Écran OLED SSD1306 128x64 (SCL)	GPIO 22	Bus I2C.
Écran OLED (alimentation)	3V3 et GND	Module à 4 broches : GND, VCC, SCL, SDA.

6. Scénario de test manuel

Après le téléversement, chaque appui sur le bouton envoie le message suivant de la liste. Le cinquième est volontairement invalide, pour tester la gestion d'erreur ; le cycle recommence ensuite.

Appui	Message	OLED	DEL	État obtenu
1	Meeting at 2 PM - Room B	Icône info, titre INFO	Éteinte	DISPLAYING
2	ALERT: evacuation required	Icône alerte, titre ALERT, cadre	Allumée	DISPLAYING
3	Coffee is ready	Icône info, titre INFO	Éteinte	DISPLAYING
4	ALERT: network failure	Icône alerte, titre ALERT, cadre	Allumée	DISPLAYING
5	(vide, invalide)	Icône erreur, « Invalid message »	Éteinte	ERROR, puis IDLE après 3 s
Aucun (10 s)	—	Écran d'attente (READY)	Éteinte	IDLE

7. Utilisation de l'intelligence artificielle

L'intelligence artificielle a été utilisée uniquement sur la branche **feature/oled-display** pour l'implémentation de l'affichage sur l'écran OLED SSD1306 128×64 en I2C, avec des icônes.
Pour réaliser cette tâche, j'ai fourni à l'IA ma classe Display, ainsi que le contexte général du projet et les fonctionnalités attendues. À partir de ces informations, elle m'a proposé une version de la classe utilisant les bibliothèques Adafruit SSD1306 et Adafruit GFX, permettant d'afficher du texte, des éléments graphiques et des icônes sur l'écran.

Prompts utilisés :

1- *À partir de ma classe Display et du contexte de mon projet ESP32, propose une implémentation permettant de gérer un écran OLED SSD1306 de 128 × 64 pixels utilisant le protocole I2C. Respecte la structure de ma classe et utilise les bibliothèques Adafruit SSD1306 et Adafruit GFX.*

2- *Propose une fonction qui découpe un message d'au maximum 60 caractères en trois lignes d'environ 20 caractères, en privilégiant les espaces pour éviter de séparer les mots. Explique comment gérer les mots trop longs et les limites de l'écran.*

3- *Propose des méthodes d'affichage pour présenter un écran d'attente, un message normal, une alerte et une erreur. Ajoute des icônes simples adaptées à chaque situation et veille à ce que le contenu soit lisible sur un écran de 128 × 64 pixels.*

Liens des ressources techniques consultées :
https://github.com/adafruit/Adafruit_SSD1306/blob/master/Adafruit_SSD1306.cpp
https://github.com/adafruit/Adafruit-GFX-Library/blob/master/Adafruit_GFX.cpp
https://adafruit.github.io/Adafruit-GFX-Library/html/class_adafruit___g_f_x.html